

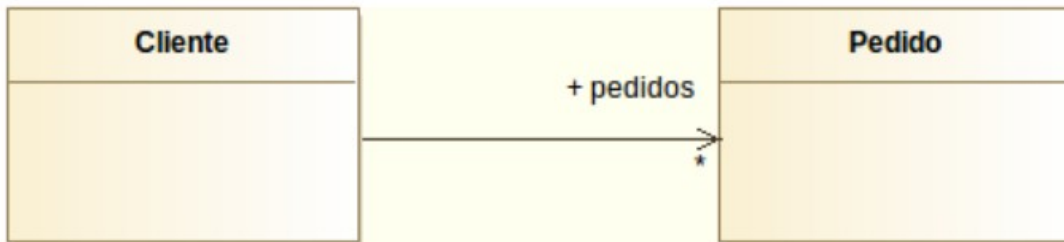
Relaciones en diagramas de clases

Asociación: unidireccional

Una asociación se da cuando una clase tiene un atributo de tipo otra clase. Es unidireccional cuando solo una clase conoce a la otra: se representa con una línea con flecha, y la misma relación se puede modelar en cualquiera de las dos direcciones.

Cliente conoce a Pedido

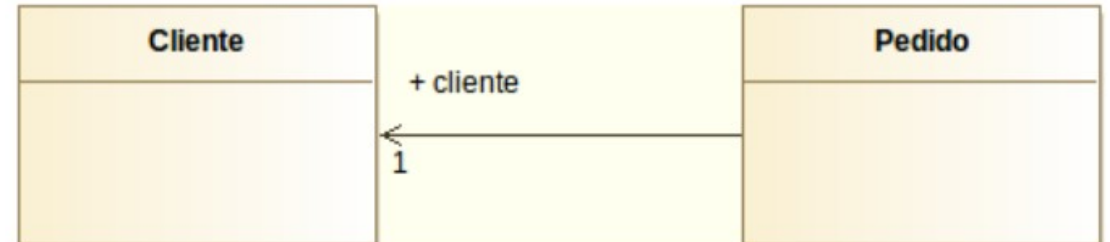
```
public class Cliente {  
    public Vector<Pedido> pedidos;  
}  
public class Pedido {  
    // NO guarda al cliente  
}
```



Flecha de Cliente a Pedido: rol "pedidos"

Pedido conoce a Cliente

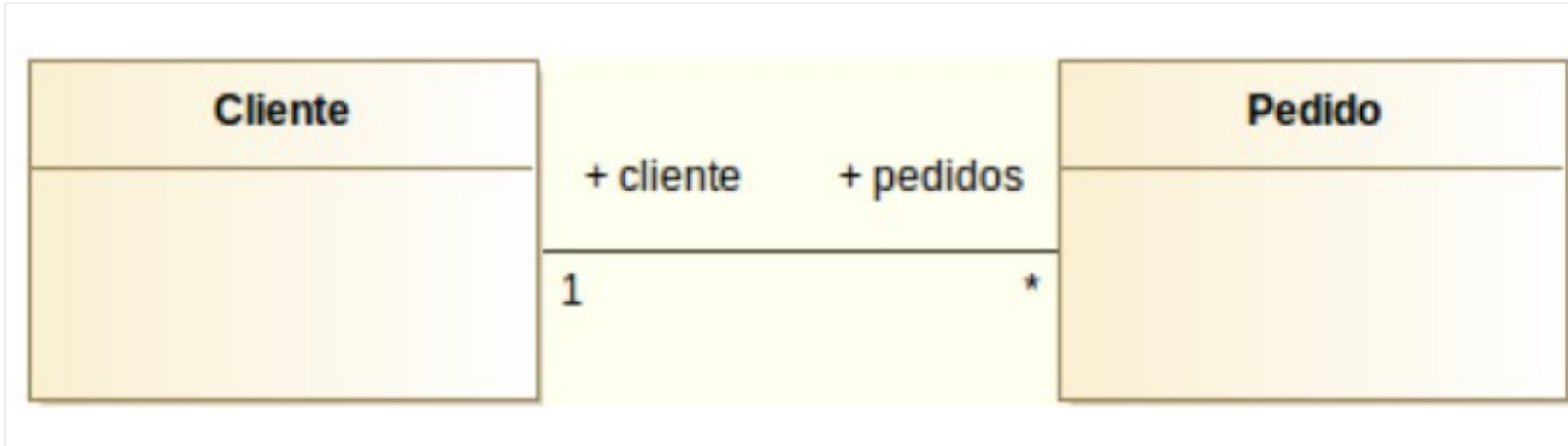
```
public class Cliente {  
    // NO guarda los pedidos  
}  
public class Pedido {  
    public Cliente cliente;  
}
```



Flecha de Pedido a Cliente: rol "cliente"

Asociación: bidireccional y roles

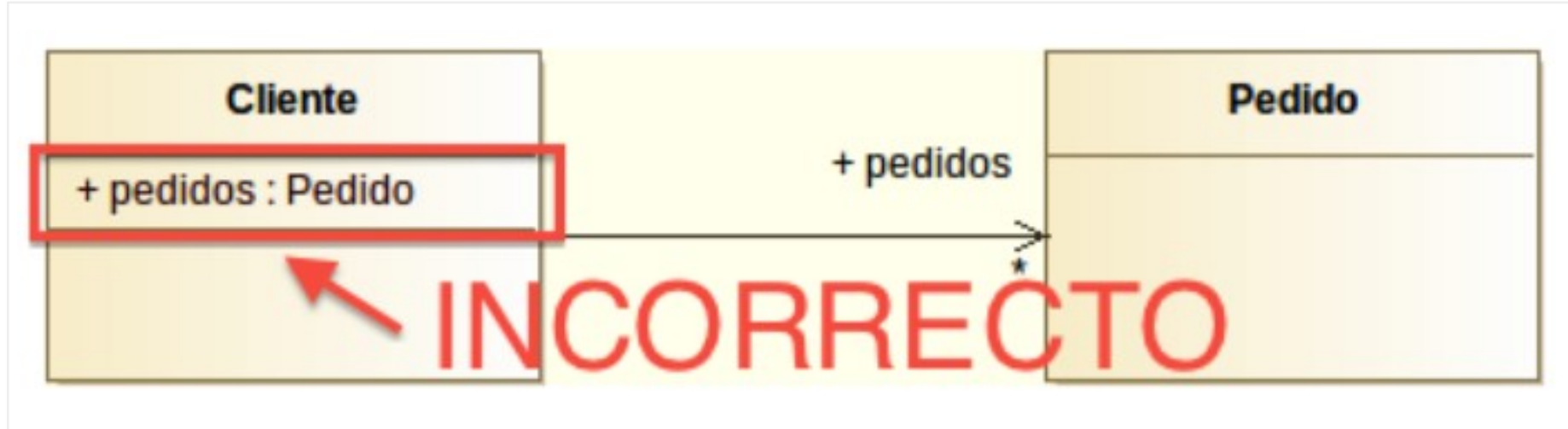
Bidireccional: ambas clases se conocen mutuamente (línea sin flechas). El rol es el nombre que toma una clase dentro de la relación —equivale al atributo en Java— y se escribe en el extremo opuesto a quien lo contiene.



Compárala con las dos anteriores: aquí van los dos roles a la vez, y no hay flecha en ningún extremo

Asociación: error frecuente

Si ya has dibujado la relación con su rol, no vuelvas a añadir ese atributo dentro de la clase: la relación ya lo representa.



Incorrecto: "pedidos" repetido como atributo y como rol de la asociación

Asociación: roles y multiplicidad en DIA

Doble clic sobre la línea de asociación para abrir sus propiedades: dos columnas, Side A y Side B, permiten fijar el rol y la multiplicidad de cada extremo.

Propiedades: UML - Association

Nombre:

Dirección:

Mostrar dirección:

Tipo:

	Side A	Side B
Rol	libro	usuarios
Multiplicidad	0..1	*
Visibilidad	Implementación	Implementación
Mostrar flecha	No	No

Color de texto:

Color de línea:

Side A / Side B: rol, multiplicidad y si se muestra la flecha en cada lado

Asociación: multiplicidad

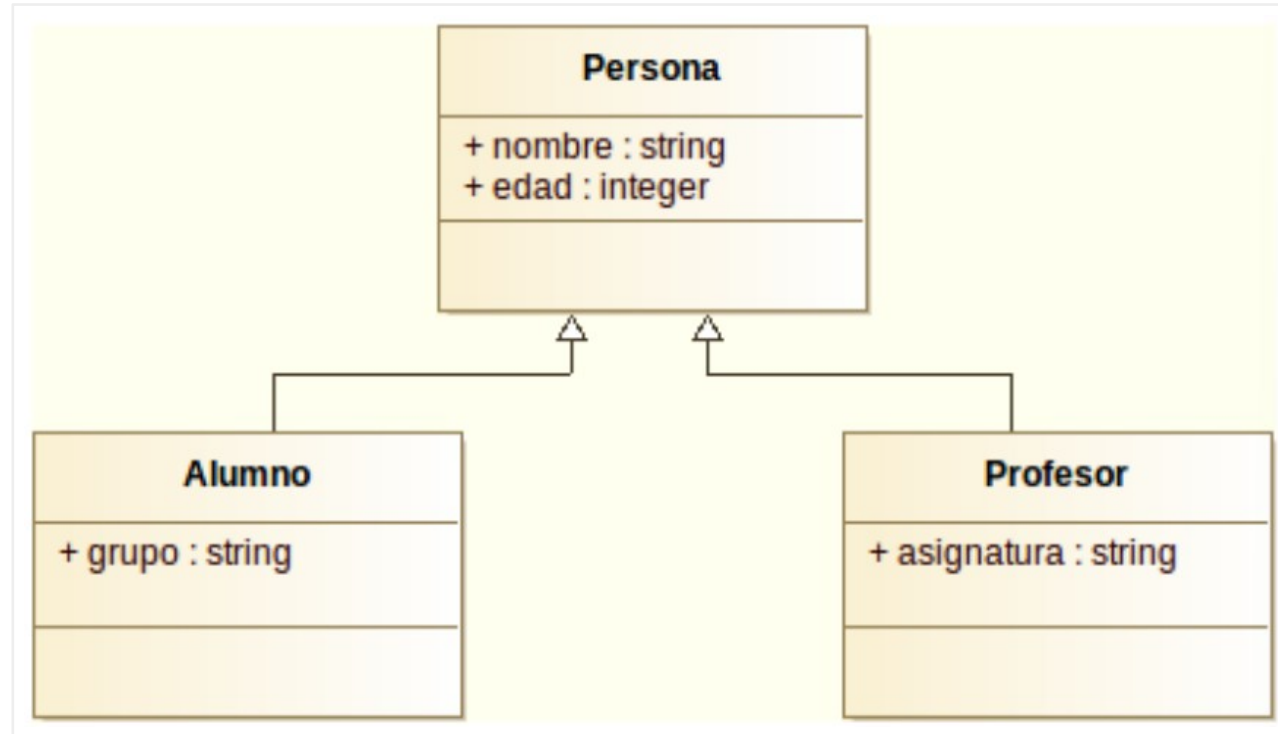
Indica cuántos objetos de una clase pueden relacionarse con los de la otra, y se traduce directamente en cómo declaras el atributo en Java.

Notación	Significado	Cómo se traduce en Java
1 / 1..1	Exactamente uno	<code>private Pelicula pelicula;</code>
0..1	Cero o uno	<code>private Pelicula pelicula; (puede ser null)</code>
* / 0..*	Cero o más, sin límite	<code>private List<Pelicula> peliculas;</code>
0..3	Entre cero y tres	<code>List<Pelicula> peliculas; // máx. 3</code>
2..3	Entre dos y tres	<code>List<Pelicula> peliculas; // 2 a 3</code>

La cardinalidad N o M no existe en UML — eso es del modelo Entidad-Relación. En UML siempre se usa * para "muchos".

Generalización (herencia)

Una clase hija hereda los atributos y métodos de la clase padre: "ClaseB es un tipo de ClaseA". Se representa con línea sólida y flecha triangular hueca, desde la hija hacia el padre.

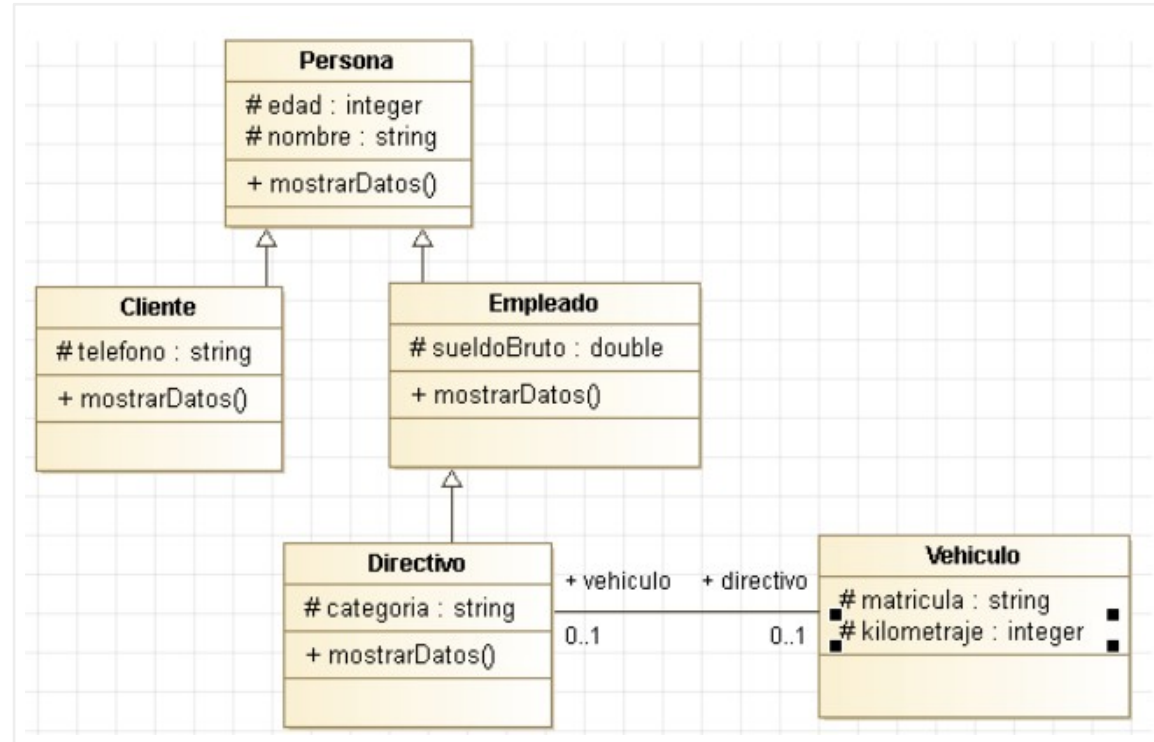


Alumno y Profesor heredan nombre y edad de Persona, y añaden solo lo propio

Si un atributo está en la clase padre, no lo repitas en las hijas: lo heredan automáticamente.

Herencia multinivel y clases abstractas

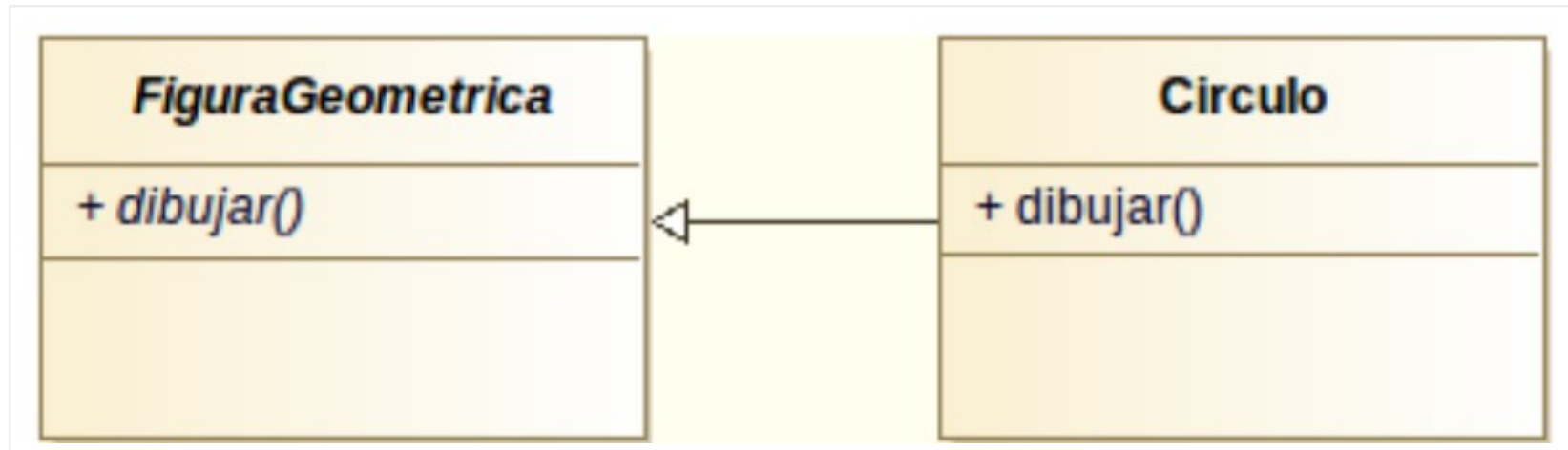
La herencia se puede encadenar en más de un nivel



Persona es padre de Cliente y Empleado; Empleado es a su vez padre de Directivo, que además se asocia con Vehiculo

Clases y métodos abstractos

Una clase abstracta tiene al menos un método sin implementar y no se puede instanciar directamente. En UML, la clase y sus métodos abstractos se escriben en cursiva.



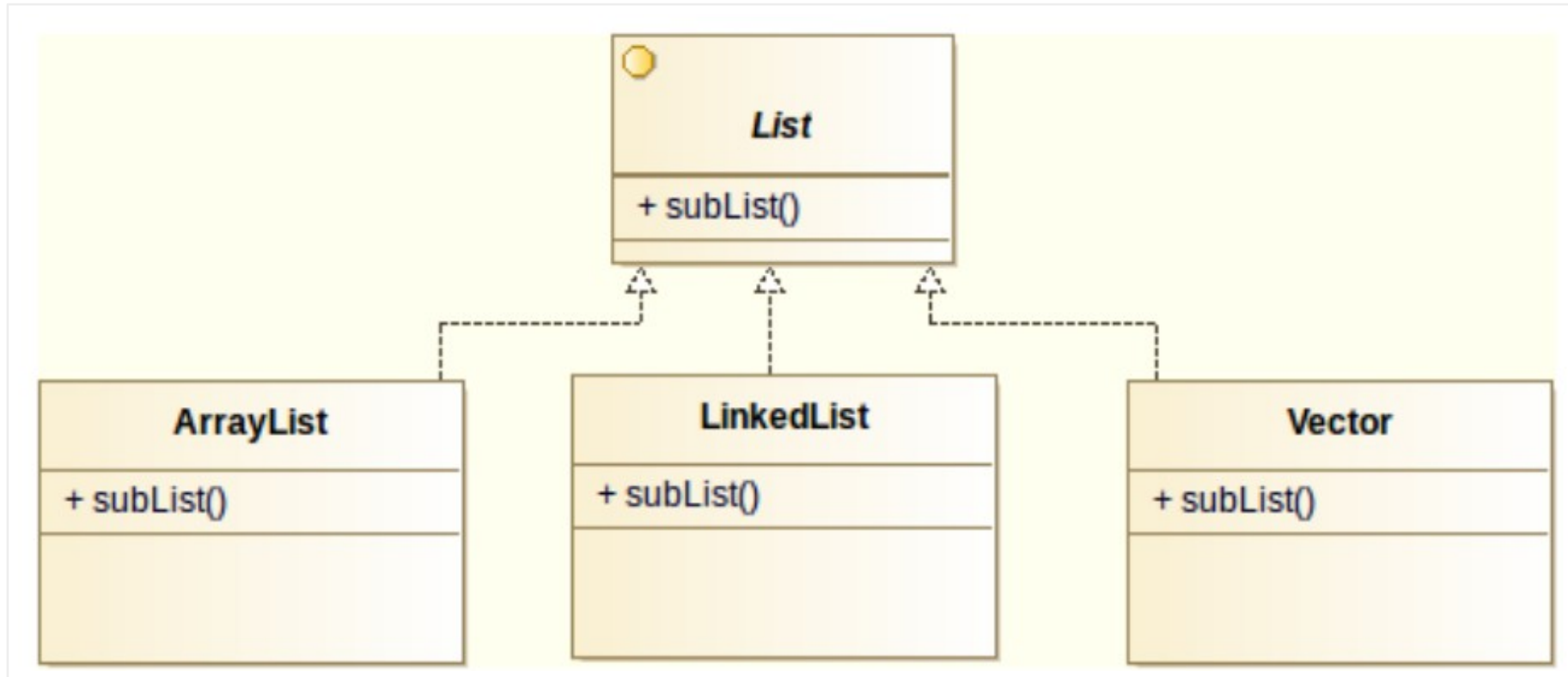
FiguraGeometrica y dibujar() en cursiva (abstractos); Circulo implementa dibujar()

```
// FiguraGeometrica.java
abstract class FiguraGeometrica {
    abstract void dibujar();
}
```

```
// Circulo.java
class Circulo extends
    FiguraGeometrica {
    void dibujar() { ... }
}
```

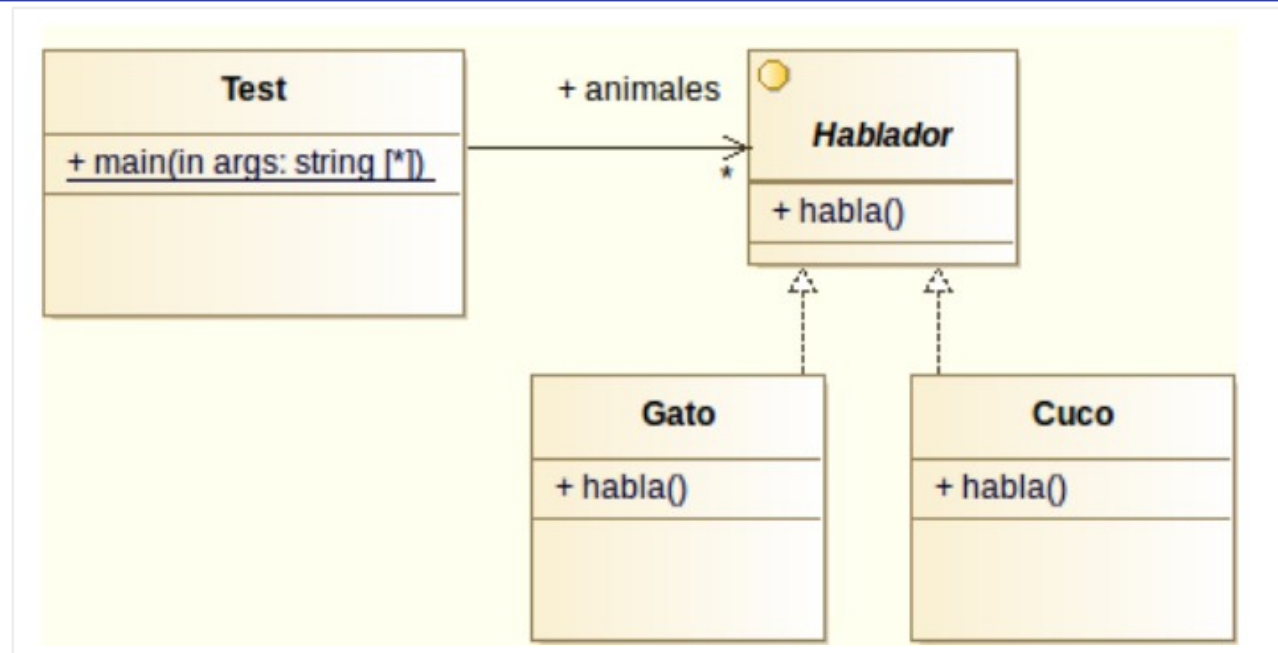
Realización (interfaces)

Una interfaz define qué métodos debe tener una clase, sin implementarlos (en Java, implements). Se representa con línea discontinua y flecha triangular hueca, desde quien implementa hacia la interfaz.



ArrayList, LinkedList y Vector realizan la interfaz List, cada una a su manera

Realización: interfaz Hablador

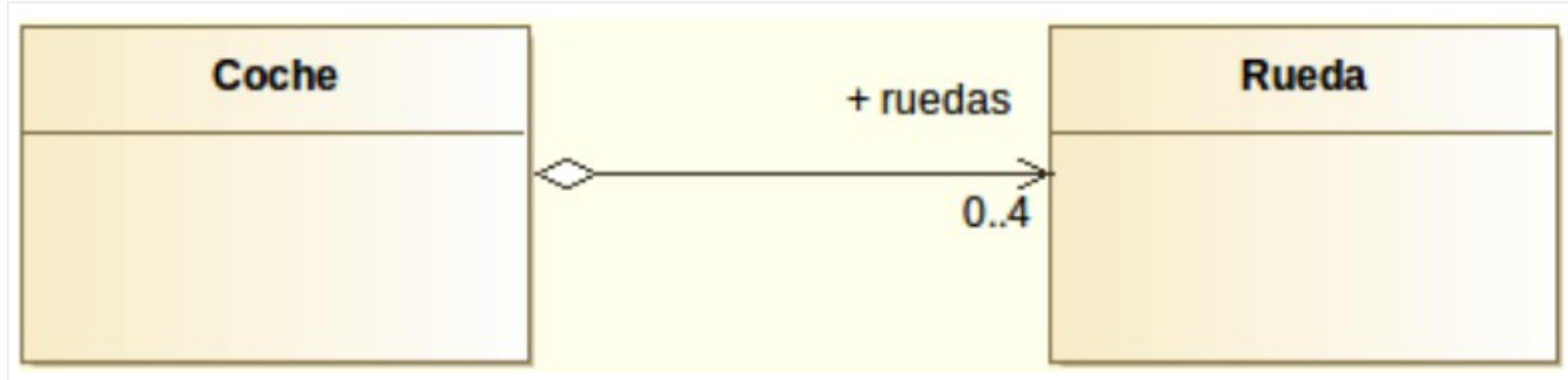


Test guarda una lista de Hablador; Gato y Cuco realizan la interfaz cada uno a su modo

```
interface Hablador { void habla(); }
class Gato implements Hablador { void habla() { println("Miau"); } }
class Cuco implements Hablador { void habla() { println("Cu cu"); } }
// Test recorre la lista de Hablador y llama habla() sin distinguir el tipo real: polimorfismo
```

Agregación

Relación "todo–parte" donde las partes pueden existir sin el todo. El rombo (hueco) va siempre en el lado del todo.

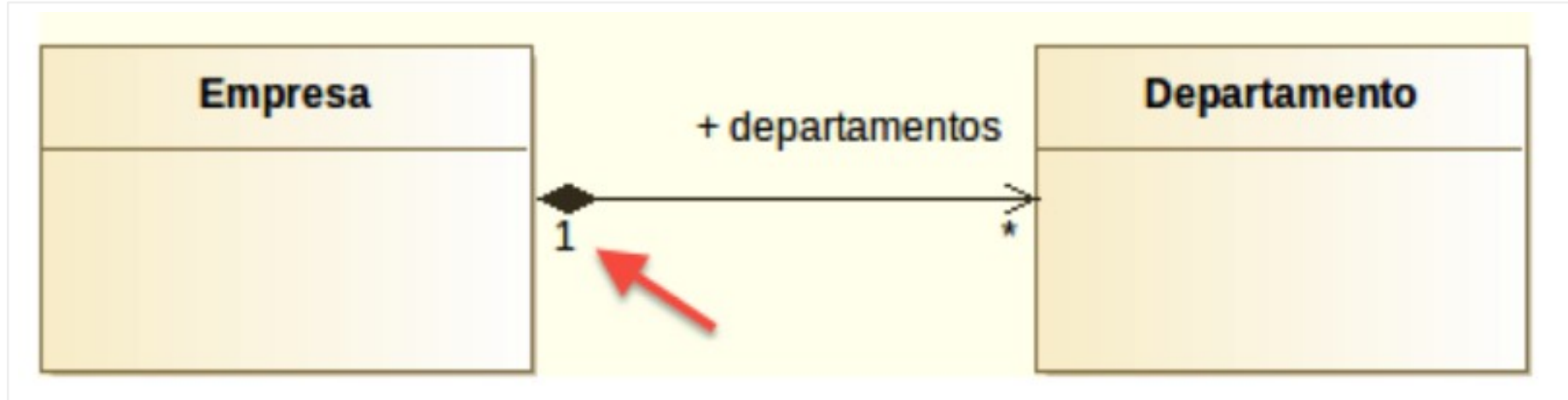


Coche agrega Ruedas: si las desmontas, siguen existiendo como piezas sueltas

En DIA, el icono de agregación es un rombo hueco en la paleta UML

Composición

Igual que la agregación, pero las partes no pueden existir sin el todo. Rombo relleno en el lado del todo.

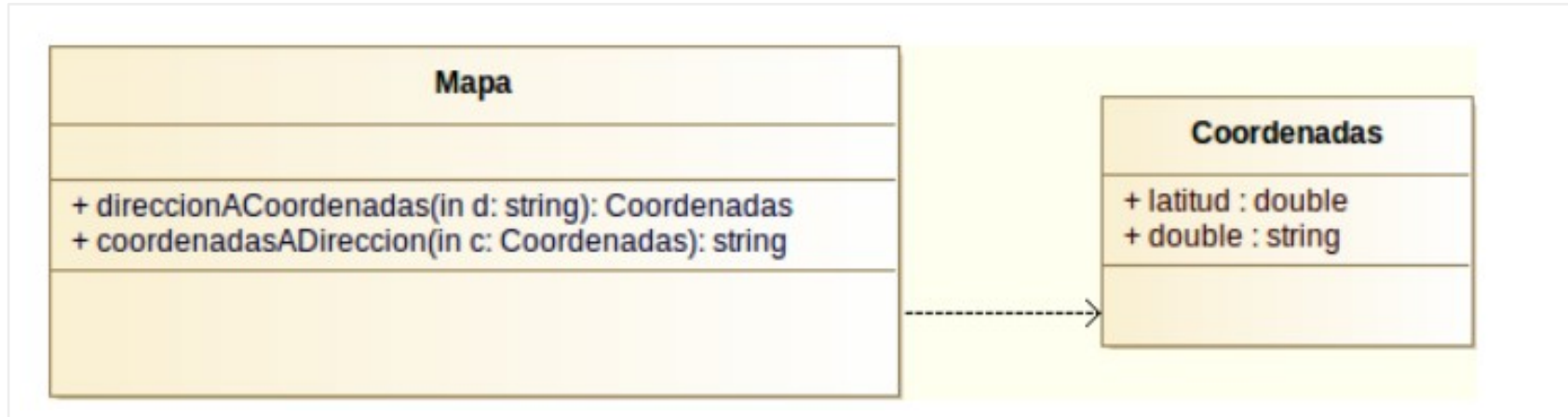


Empresa compone Departamentos: si la empresa desaparece, ellos desaparecen con ella

Tanto el rombo de agregación como el de composición van en el lado del todo, no en el de la parte. DIA solo tiene el icono de Agregación: para Composición, cambia el campo Tipo en las propiedades de la línea.

Dependencia

Una clase usa a otra de forma temporal (parámetro, variable local o retorno), sin guardar una referencia permanente. Línea discontinua con flecha en "V".

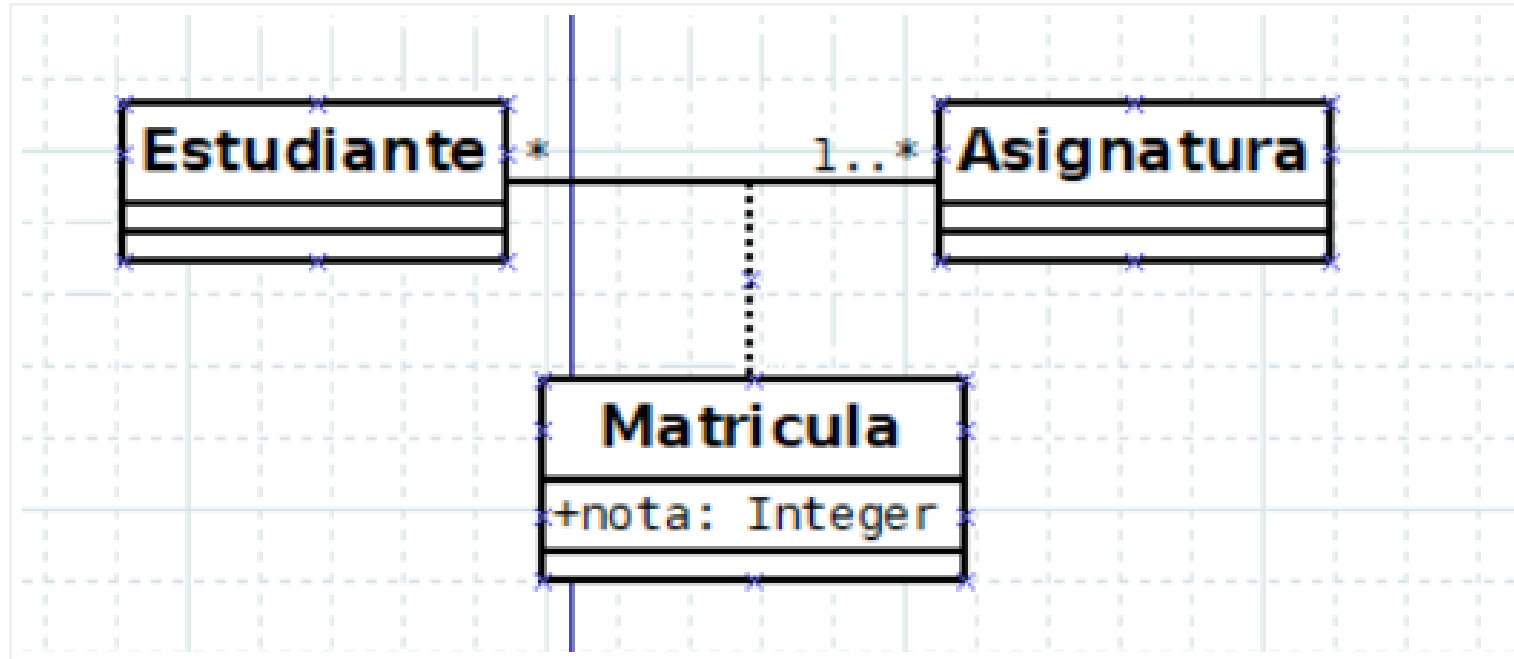


Mapa usa Coordenadas como parámetro y retorno, pero no la guarda como atributo

```
public Coordenadas direccionACoordenadas(String d) { ... }
public String coordenadasADireccion(Coordenadas c) { ... }
// Ninguno de los dos guarda Coordenadas en un atributo de Mapa
```

Clase asociativa

Cuando una relación tiene sus propios atributos o métodos, se representa como una clase unida a la línea de asociación con una línea discontinua.



Matricula no pertenece solo a Estudiante ni solo a Asignatura: pertenece a su relación

En DIA no hay un elemento específico: se dibuja la clase aparte y se une a la línea de asociación con la herramienta de línea estándar, en estilo discontinuo.

¿Generalización o realización?

	Generalización (herencia)	Realización (interfaz)
Palabra clave Java	extends	implements
¿Cuántas puede tener?	Solo una clase padre	Múltiples interfaces
¿La "padre" implementa?	Sí, puede tener código	No, solo declara la firma
Cuándo usarla	La hija es un tipo de la padre	La clase promete comportarse según el contrato

Si necesitas que una clase tenga comportamiento de varios tipos distintos, usa interfaces: Java solo permite heredar de una clase padre, pero puedes implementar tantas interfaces como quieras.

¿Clase abstracta o interfaz?

	Clase abstracta	Interfaz
Atributos de instancia	Sí	No (solo static final)
Métodos con implementación	Sí	No (en Java < 8)
Una clase puede extender/implementar	Solo una	Varias
Relación UML	Generalización (línea sólida)	Realización (línea discontinua)

Elige clase abstracta cuando las subclases comparten código ya implementado. Elige interfaz cuando solo quieras garantizar que un comportamiento existe, sin importar cómo se implementa.